AUTONOMOUS SYSTEMS

Argumentation Engine

and Strategic Argumentation

Submitted by

Sarah Fatima	B23EE1093
Sahil Narkhede	B23CS1060
Namya Dhingra	B23CS1040
Anshit Agarwal	B23CS1087

12 November 2026

Pure Python implementation; reproducible with the standard library.

Abstract

This project implements a complete small-framework engine for Dung-style abstract argumentation and studies strategic reporting under the grounded direct argumentation mechanism. The framework engine computes admissible and complete sets, grounded, preferred, stable, and semi-stable extensions, legal in/out/undec labellings, and acceptance statuses. The strategic component evaluates reports, enumerates best responses, checks the truthfulness inequality of Theorem 5.3, and tests the odd-path condition of Theorem 5.4, including two explicit interpretations of paths in cyclic graphs.

The implementation is validated against the book's Examples 5.10 and 5.11, hand-constructed semantic cases, and randomized cross-checks. The experiments evaluate 1,000 random instances for each of three attack probabilities. For the main no-self-attack setting, the measured strategyproof fractions are 0.632, 0.317, and 0.126 for $p = 0.15, 0.25, 0.35$, respectively. No sampled instance satisfied the Theorem 5.4 condition while failing strategyproofness. Conversely, many instances remained strategyproof even though the sufficient condition failed, illustrating that the condition is not necessary.

Contents

1 Overview and objectives **1**

1.1 Assignment coverage . . . 1

2 Argumentation framework and semantics **1**

2.1 Definitions . . . 1

2.2 Labellings and acceptance . . . 2

2.3 Implementation approach . . . 2

3 Strategic argumentation **2**

3.1 Model and utility . . . 2

3.2 Mechanism and best response . . . 3

3.3 Theorem 5.3: truthfulness criterion . . . 3

3.4 Theorem 5.4: odd paths . . . 3

4 Worked examples from the book **3**

4.1 Example 5.10: hiding can increase utility . . . 3

4.2 Example 5.11: sufficient is not necessary . . . 4

5 Experiments **4**

5.1 Protocol . . . 4

5.2 Strategyproofness rates . . . 4

5.3 Theorem 5.4 condition versus strategy proofness . . . 5

5.4 Why can the condition fail while truth remains optimal? . . . 5

5.5 A profitable deviation . . . 6

6 Verification and demonstration **6**

6.1 Automated tests 6
6.2 Demonstration plan 7

7 Limitations and interpretation **7**

8 Reproduction and project layout **7**

9 Conclusion **7**

1 Overview and objectives

An abstract argumentation framework represents arguments and directed attacks between them. The first objective is to provide a reusable implementation of the principal Dung semantics. The second objective is to implement the grounded direct argumentation mechanism and investigate whether an agent can improve its outcome by concealing arguments it owns.

1.1 Assignment coverage

Requirement	Implementation
Framework and basic semantics	`af_project/argumentation.py`: framework class, conflict-freeness, defence, characteristic function, admissibility, completeness, grounded and other extensions, labellings, acceptance.
Strategic mechanism and best responses	`af_project/strategic.py`: induced revealed framework, grounded outcome, utility, and exhaustive best-response enumeration.
Theorems 5.3 and 5.4	Two Theorem 5.3 checks, plus odd-path detection and witnesses for Theorem 5.4.
Book examples and errata	`af_project/book_examples.py` and the tests/demo cover Examples 5.10 and 5.11 and both assignment-noted misprints.
Experiments and explanation	`af_project/experiments.py`, `run_experiments.py`, `analyse.py`, and saved results in `af_project/results/`.
Submission demonstration	`af_project/demo.py`, `demo_output.txt`, and `DEMO.md`.

Table 1: Where each requested component is implemented.

2 Argumentation framework and semantics

2.1 Definitions

An argumentation framework is a pair

$$AF = (A, R),$$

where A is a finite set of arguments and $R \subseteq A \times A$ is the attack relation. We write aRb when a attacks b. For $S \subseteq A$, define the set attacked by S as

$$S^+ = \{b \in A : \exists a \in S,\ (a, b) \in R\}.$$

The set of attackers of an argument a is $\mathrm{Att}(a) = \{b \in A : (b, a) \in R\}$.

Conflict-free S is conflict-free iff no attack has both endpoints in S; self-attacks therefore make a set containing that argument non-conflict-free.

Defence S defends a iff each attacker of a is attacked by at least one member of S. An unattacked argument is defended by every set, including the empty set.

Characteristic function $F(S) = \{a \in A : S \text{ defends } a\}$.

Admissible S is conflict-free and $S \subseteq F(S)$.

Complete S is conflict-free and $S = F(S)$.

Grounded The grounded extension is the least fixed point of F, computed by iterating from the empty set.

Preferred An inclusion-maximal admissible set.

Stable A conflict-free set S with $S \cup S^+ = A$.

Semi-stable An admissible set whose range $S \cup S^+$ is inclusion-maximal among admissible sets.

The characteristic function is monotone. Starting from $\varnothing$, iteration reaches its least fixed point on a finite framework. The grounded extension is unique and is contained in every complete extension.

2.2 Labellings and acceptance

A labelling assigns one of *in*, *out*, or *undec* to each argument. It is legal if, for every argument a:

$$\begin{aligned} a \text{ is in} &\iff \text{every attacker of } a \text{ is out,} \\ a \text{ is out} &\iff \text{at least one attacker of } a \text{ is in,} \\ a \text{ is undec} &\iff \text{no attacker is in and not all attackers are out.} \end{aligned}$$

Legal labellings correspond one-to-one with complete extensions via their in-labelled arguments. Labellings without undecided arguments correspond to stable extensions.

For a selected semantics, an argument is *skeptically accepted* if it belongs to every extension, *credulously accepted* if it belongs to some but not all extensions, and *rejected* if it belongs to none. If stable semantics has no extension, the implementation reports an explicit undefined/no-extensions status instead of assigning one of these labels.

2.3 Implementation approach

The `ArgumentationFramework` object stores the argument set, attack relation, and precomputed attacker/target maps. Its constructor rejects attacks mentioning unknown arguments. Semantic computations use exhaustive subset enumeration, which is transparent and appropriate for the small frameworks in this assignment:

- up to $2^{|A|}$ candidate sets for extension semantics;
- up to $3^{|A|}$ candidate labellings;
- iteration of F for the grounded extension.

The implementation also provides function-style wrappers around the class methods, so both object-oriented and function-oriented use are available.

3 Strategic argumentation

3.1 Model and utility

There are agents $i \in N$ with pairwise disjoint true argument sets A_i. A report is any subset $R_i \subseteq A_i$. Given public attack relation R, the mechanism forms the framework induced by the

union of reports:

$$M(R_1, \ldots, R_n) = E_{\text{gr}}\left(\left(\bigcup_i R_i,\ R \cap \left(\bigcup_i R_i\right)^2\right)\right).$$

The utility of agent i is the number of its true arguments accepted:

$$u_i(R_1, \ldots, R_n) = |A_i \cap M(R_1, \ldots, R_n)|.$$

Thus an agent cannot invent an argument, and arguments it owns but hides remain part of its utility type, but not of the reported graph. Truthful reporting is weakly optimal when no alternative report gives strictly greater utility; it need not be the unique best response.

3.2 Mechanism and best response

`mechanism(attacks, reports)` returns the grounded extension of the induced revealed framework. `best_responses(agent, agents, attacks, others_reports)` checks every subset of that agent's true arguments, evaluates the resulting grounded extension, and returns both the maximum utility and all reports attaining it. This exhaustive method gives a direct, auditable best-response calculation for small types.

3.3 Theorem 5.3: truthfulness criterion

For every agent i, every subset $S \subseteq A_i$ to hide, and every possible union A_{-i} of other agents' reports, Theorem 5.3 requires

$$\left|A_i \cap E_{\text{gr}}(A_i \cup A_{-i}, R)\right| \ \geq\ \left|A_i \cap E_{\text{gr}}((A_i \setminus S) \cup A_{-i}, R)\right|. \tag{1}$$

The code checks the equivalent best-response statement with a bitmask implementation and also provides a second, literal inequality implementation using the framework class. The tests compare the two approaches on randomized instances. Inputs to the exhaustive checks are validated: ownership sets must be disjoint, and every attack must mention arguments owned by an agent.

3.4 Theorem 5.4: odd paths

An odd-length directed path from α to β is denoted $\alpha \rightsquigarrow \beta$. The sufficient condition checked here is that no agent owns two distinct arguments α, β with $\alpha \rightsquigarrow \beta$ in the full attack graph. The implementation returns both the violations and witness paths.

In cyclic graphs, the word path can be interpreted in two ways. The default *walk* interpretation permits repeated vertices; the search is breadth-first over states $(v, \ell \bmod 2)$ and terminates because there are only $2|A|$ such states. The *simple path* interpretation disallows repeated vertices and is checked separately. Reporting both makes the ambiguity visible rather than silently choosing one interpretation. The experimental headline uses the walk interpretation.

4 Worked examples from the book

4.1 Example 5.10: hiding can increase utility

The arguments are partitioned as

$$A_x = \{a_1, a_4, a_5\}, \qquad A_y = \{a_2\}, \qquad A_z = \{a_3\},$$

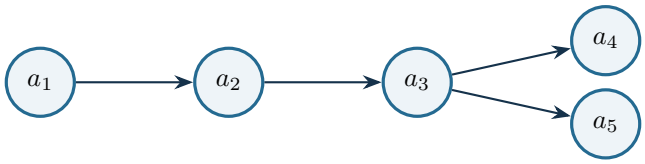


Figure 1: Attack graph for Example 5.10.

with attacks $a_1 \to a_2$, $a_2 \to a_3$, $a_3 \to a_4$, and $a_3 \to a_5$.

When all agents report truthfully, the grounded extension is $\{a_1, a_3\}$. Utilities are $(u_x, u_y, u_z) = (1, 0, 1)$. If x hides a_1, the revealed graph has grounded extension $\{a_2, a_4, a_5\}$, giving x utility 2. Truthful reporting is therefore not a best response against truthful reports by y and z, and the mechanism is not strategyproof. The odd paths $a_1 \to a_2 \to a_3 \to a_4$ and $a_1 \to a_2 \to a_3 \to a_5$ also witness failure of the Theorem 5.4 condition.

Correction of the stated misprint. Example 5.10's utility discussion swaps y's and z's argument sets. Using each agent's actual set gives $u_y = 0$ and $u_z = 1$.

4.2 Example 5.11: sufficient is not necessary

Example 5.11 adds argument a_6 to A_x and adds the attack $a_6 \to a_3$. The truthful grounded extension is $\{a_1, a_4, a_5, a_6\}$ and all four of x's arguments are accepted. The instance is strategyproof, although the odd paths from a_1 to a_4 and a_5 remain. This demonstrates that Theorem 5.4's condition is sufficient but not necessary.

Correction of the stated misprint. The argument that cancels the indirect self-defeat is a_6, not a_5. Revealing a_6 makes it accepted and attacks a_3, preventing the chain from using a_3 to defeat a_4 and a_5.

5 Experiments

5.1 Protocol

We generated instances with three agents and n uniformly selected from $\{6, 7, 8\}$. For each ordered pair of distinct arguments, an attack is independently included with probability p. Each argument is independently assigned uniformly to one of the three agents; an agent may own no arguments. We used $p \in \{0.15, 0.25, 0.35\}$ and generated 1,000 instances for each p (3,000 total), using seed 2024. Strategyproofness is tested exhaustively, not estimated by sampling deviations. A separate variant permits self-attacks.

The main experiment excludes self-attacks, matching the explicit restriction $a \neq b$ in the assignment's random attack model. The extra variant provides a sensitivity check for the ambiguous phrase "each possible attack."

5.2 Strategyproofness rates

The measured fraction declines as the attack graph becomes denser. In these small random instances, denser attack relations create more opportunities for interactions between an agent's own arguments. However, the rates depend on the assignment and sampling model; they are not universal probabilities about argumentation frameworks.

p	No self-attacks	Self-attacks allowed
0.15	632/1000 = **0.632**	535/1000 = 0.535
0.25	317/1000 = **0.317**	254/1000 = 0.254
0.35	126/1000 = **0.126**	101/1000 = 0.101

Table 2: Fraction of strategyproof instances for each attack probability.

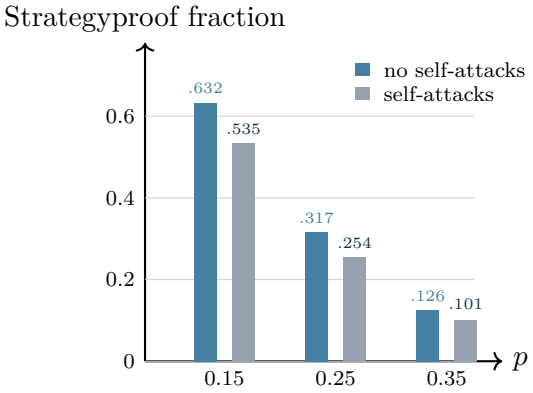


Figure 2: Measured strategyproof rates. Rates decrease as attack density increases.

5.3 Theorem 5.4 condition versus strategy proofness

p	Condition holds (walk)	Holds but not SP	SP, condition fails (walk)	SP, condition fails (simple)
0.15	118	0	514	512
0.25	31	0	286	284
0.35	4	0	122	121

Table 3: Cross-tabulation for the main, no-self-attack experiment.

No sampled instance satisfied the condition yet failed strategy proofness, consistent with Theorem 5.4's sufficient-condition claim. This finite experiment is an empirical verification, not a proof. The number of condition-satisfying instances is small at high density, so the zero count at $p = 0.35$ should be interpreted with that limitation in mind.

The condition is not necessary: 514, 286, and 122 instances at the three probabilities were strategyproof even under the default walk interpretation while violating the condition. This count includes cases where truthful reporting ties a deviation; such ties do not violate weak strategyproofness.

5.4 Why can the condition fail while truth remains optimal?

For every strategyproof instance that failed the walk condition, we compared the agent's accepted true arguments under truth and every strictly smaller report. We classify each instance by its most informative deviation:

- **Swap:** the accepted set changes but its size stays the same; the agent exchanges one accepted argument for another.
- **Loss-only:** at least one deviation reduces utility, and no equal-utility swap occurs.

- **Inert:** every deviation leaves the accepted set unchanged.

p	Strategyproof, condition fails	Swap	Loss-only	Inert
0.15	514	445	69	0
0.25	286	243	43	0
0.35	122	103	19	0

Table 4: Classification of strategyproof instances that violate the walk condition.

Most such instances contain a zero-sum swap. A short odd path can make it possible to release a same-agent argument by hiding another, but hiding also removes an argument from the reported framework. For example, with a single attack $\alpha \to \beta$ and both arguments owned by one agent, truth accepts α and rejects β. Hiding α can accept β instead, leaving the utility unchanged. This violates the sufficient condition but does not make hiding profitable.

In a loss-only example, redundant blockers can prevent the hidden argument's removal from releasing a target. If two accepted attackers block an argument owned by the agent, hiding just one still leaves the other attack in place; hiding both can cost more utility than it gains. The condition tracks odd-path structure, not the net utility of every possible report, which explains why it can fail without a profitable deviation.

5.5 A profitable deviation

The smallest illustrative mechanism in the demonstration has agent 1 owning $\{\alpha, \beta\}$ and agent 2 owning $\{d, e\}$. The attacks are $\alpha \to \beta$, $\alpha \to d$, $d \to e$, and $e \to \alpha$. The truthful grounded outcome leaves agent 1 with utility 0; hiding α breaks the odd cycle and allows β to be accepted, giving utility 1. This is a concrete case where the condition fails and the mechanism is not strategyproof.

6 Verification and demonstration

6.1 Automated tests

The standard-library test suite is `af_project/tests.py`. It checks:

- hand-constructed chains, even and odd cycles, self-attacks, and examples distinguishing semantics;
- the book's Examples 5.10 and 5.11, including the assignment's corrections;
- 300 random frameworks for structural cross-checks, including grounded-as-least-complete and the correspondence between complete extensions and legal labellings;
- the bitmask and class-based grounded mechanism implementations on random instances;
- consistency between best-response enumeration and the truthfulness checker;
- agreement between the two independent Theorem 5.3 implementations on 150 randomized instances;
- odd-walk search against bounded brute-force enumeration on 100 random graphs, plus witness validity;

- the Theorem 5.4 sufficient-condition implication on randomized checks.

The test command completed successfully with the final output `ALL TESTS PASSED`.

6.2 Demonstration plan

The live demonstration script `af_project/demo.py` and guide `af_project/DEMO.md` provide a concise walkthrough:

1. classic frameworks and their extensions and labellings;
2. Example 5.10, the profitable concealment, and the Theorem 5.3/5.4 checks;
3. Example 5.11, demonstrating that the Theorem 5.4 condition is not necessary;
4. examples where the condition holds and fails, and walk-versus-simple-path behavior on cycles.

7 Limitations and interpretation

- The semantics and strategic checks are exhaustive and exponential. They are intended for the small frameworks in this assignment, not large-scale deployment.
- The random experiment uses a specific model: uniformly chosen size from 6–8, independent assignment of each argument to one of three agents, and independent attacks. Agents may receive zero arguments.
- The path definition on cyclic graphs has two plausible interpretations. Both are implemented and reported; the main condition uses walks, which allow repeated vertices.
- Randomized experiments support reproducibility and reveal patterns, but cannot establish a theorem. Theorem 5.4's sufficiency is supported by its statement and checked exhaustively per sampled instance; this is not a formal proof.
- Strategyproofness is weak: truth must be among the best reports, not the unique best report.

8 Reproduction and project layout

The project uses Python 3 and the standard library only. From the repository root:

```
cd af_project
python tests.py
python demo.py
python run_experiments.py
```

The experiment runner accepts an optional instance count per probability. For example, `python run_experiments.py 1000` runs 1,000 instances per p and writes result tables and JSON data under `af_project/results/`. Results use a fixed seed of 2024.

9 Conclusion

The project delivers both requested parts: a working implementation of standard argumentation semantics and an exhaustive small-instance study of strategic reporting under the grounded

Path	Purpose
`af_project/argumentation.py`	Argumentation framework and semantics.
`af_project/strategic.py`	Strategic mechanism, best responses, theorem checks.
`af_project/book_examples.py`	Encoded book examples and corrected notes.
`af_project/tests.py`	Automated assertions and randomized cross-checks.
`af_project/demo.py`	Runnable demonstration.
`af_project/experiments.py`	Random instance generation and summaries.
`af_project/results/`	Reproducible experiment summaries and instance records.
`README.md`	Project overview and usage instructions.

Table 5: Files included with the submission.

mechanism. The implementation passes its tests and reproduces the book examples, including the specified corrections. In the experiments, strategyproofness becomes less common as attack density increases, while the Theorem 5.4 condition remains sufficient but is far from necessary. In most sampled strategyproof instances that violate the condition, concealment merely trades one accepted argument for another rather than increasing utility.